

深圳大学实验报告

课程名称：Java 程序设计

实验项目名称：课程实验 3：常用集合类和线程

学院：数学科学学院

专业：信息与计算科学（数学与计算机实验班）

指导教师：潘微科

报告人：黄嘉聪 学号：2023192044 班级：1

实验时间：2024 年 11 月 1 日（周五）-2024 年 11 月 20 日（周三）

实验报告提交时间：2024 年 11 月 20 日

教务部制

实验目的与要求:

实验目的: 掌握常用的集合类, 能够较为熟练的查阅 Java 提供的常见的类, 并进行程序设计, 掌握 Java 程序设计中的线程同步等技术。

实验要求:

Part 1 (25 分)

(1.1). 编写 Java 应用程序, 实现浮点数 (float) 稀疏矩阵的乘法和加法运算, 其中稀疏矩阵是指矩阵中的绝大部分元素的值为 0。在命令行读入和输出矩阵中的元素的时候, 采用三元组的方式, 即行号、列号和数值, 例如“第 2 行、第 3 列、数值 3.2”表示为 2 3 3.2。要求以 1 个 3*4 的矩阵 ([1.2 0 0 0; 0 0 3.1 0; 0 0 0 2.2]) 和一个 4*5 的矩阵 ([0 1.1 1.5 0 0; 0 0 3.2 0 0; 0 1.3 0 0 -3.2; -1.0 6.2 0 0 0]) 相乘, 以及 1 个 3*4 的矩阵 ([1.2 0 0 0; 0 0 3.1 0; 0 0 0 2.2]) 和 1 个 3*4 的矩阵 ([1.2 -3 -5.3 0; 0 1 0.1 -0.4; 2 2 1 0.2]) 相加为例, 在报告中附上程序截图、完整的运行结果截图和简要文字说明。(10 分)

(1.2). 有 12 个国家 (美国、中华人民共和国、德国、日本、英国、印度、法国、意大利、加拿大、韩国、以色列、俄罗斯), 其属性有 name、GDP2023 和 Olympics2024, 分别表示国家名称、世界银行公布的 2023 年的国内生产总值 (单位: 百万美元) 和在 2024 年巴黎奥运会上获得的奖牌数量。

编写一个 Java 应用程序, 要求使用 TreeSet。(i) 按照 Olympics2024 从大到小排序输出这些国家的信息;(ii) 按照 GDP2023 从大到小排序输出这些国家的信息。要求以上 (i) 和 (ii) 两小题都通过以下两种方式实现: 通过实现 Comparator 接口或通过实现 Comparable 接口。在报告中附上程序截图、完整的运行结果截图和详细的文字说明。(15 分)

Part 2 (25 分)

(2.1) 将第 8 章讲义 (JavaPD-Ch08) 中的 5 个应用程序 (Example8_1, Example8_2, Example8_3, Example8_4, Example8_6) 在 Eclipse 中运行, 如运行结果不唯一, 则需要运行多次并至少得到两个不同的结果。对重要语句加上注释。在报告中附上程序截图、运行结果截图和简要文字说明 (对运行结果做出解释)。(5 分)

(2.2). 运行以下三个程序 (每个程序运行 5 次), 并对输出结果给出分析。在报告中附上程序截图和简要的文字说明 (包括对结果的分析)。(10 分)

程序 1:

```

// The task for printing a character a specified number of times
class PrintChar implements Runnable {
    private char charToPrint; // The character to print
    private int times; // The number of times to repeat

    /** Construct a task with a specified character and number of
     *  times to print the character
     */
    public PrintChar(char c, int t) {
        charToPrint = c;
        times = t;
    }

    @Override /** Override the run() method to tell the system
     *  what task to perform
     */
    public void run() {
        for (int i = 0; i < times; i++) {
            System.out.print(charToPrint);
        }
    }
}

// The task class for printing numbers from 1 to n for a given n
class PrintNum implements Runnable {
    private int lastNum;

    /** Construct a task for printing 1, 2, ..., n */
    public PrintNum(int n) {
        lastNum = n;
    }

    @Override /** Tell the thread how to run */
    public void run() {
        for (int i = 1; i <= lastNum; i++) {
            System.out.print(" " + i);
        }
    }
}

public class TaskThreadDemo {
    public static void main(String[] args) {
        // Create tasks
        Runnable printA = new PrintChar('a', 100);
        Runnable printB = new PrintChar('b', 100);
        Runnable print100 = new PrintNum(100);

        // Create threads
        Thread thread1 = new Thread(printA);
        Thread thread2 = new Thread(printB);
        Thread thread3 = new Thread(print100);

        // Start threads
        thread1.start();
        thread2.start();
        thread3.start();
    }
}

```

程序 2:

```

// The task for printing a character a specified number of times
class PrintChar implements Runnable {
    private char charToPrint; // The character to print
    private int times; // The number of times to repeat

    /** Construct a task with a specified character and number of
     * times to print the character
     */
    public PrintChar(char c, int t) {
        charToPrint = c;
        times = t;
    }

    @Override /** Override the run() method to tell the system
     * what task to perform
     */
    public void run() {
        for (int i = 0; i < times; i++) {
            System.out.print(charToPrint);
        }
    }
}

// The task class for printing numbers from 1 to n for a given n
class PrintNum implements Runnable {
    private int lastNum;

    /** Construct a task for printing 1, 2, ..., n */
    public PrintNum(int n) {
        lastNum = n;
    }

    @Override /** Tell the thread how to run */
    public void run() {
        for (int i = 1; i <= lastNum; i++) {
            System.out.print(" " + i);
        }
    }
}

import java.util.concurrent.*;

public class ExecutorDemo {
    public static void main(String[] args) {
        // Create a fixed thread pool with maximum three threads
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Submit runnable tasks to the executor
        executor.execute(new PrintChar('a', 100));
        executor.execute(new PrintChar('b', 100));
        executor.execute(new PrintNum(100));

        // Shut down the executor
        executor.shutdown();
    }
}

```

程序 3:

```
import java.util.concurrent.*;

public class AccountWithoutSync {
    private static Account account = new Account();

    public static void main(String[] args) {
        ExecutorService executor = Executors.newCachedThreadPool();

        // Create and launch 100 threads
        for (int i = 0; i < 100; i++) {
            executor.execute(new AddAPennyTask());
        }

        executor.shutdown();

        // Wait until all tasks are finished
        while (!executor.isTerminated()) {
        }

        System.out.println("What is balance? " + account.getBalance());
    }

    // A thread for adding a penny to the account
    private static class AddAPennyTask implements Runnable {
        public void run() {
            account.deposit(1);
        }
    }

    // An inner class for account
    private static class Account {
        private int balance = 0;

        public int getBalance() {
            return balance;
        }

        public void deposit(int amount) {
            int newBalance = balance + amount;

            // This delay is deliberately added to magnify the
            // data-corruption problem and make it easy to see.
            try {
                Thread.sleep(5);
            }
            catch (InterruptedException ex) {
            }

            balance = newBalance;
        }
    }
}
```

(2.3) 第 8 章讲义 (JavaPD-Ch08) 中的第 5 个应用程序 (Example8_5) 存在线程间不同步的问题, 请修改该程序, 以解决不同步的问题。在报告中附上程序截图、运行结果截图和详细的文字说明 (包括设计的思路和合理性分析)。(10 分)

Part 3 (30 分)

(3.1). 编写 Java 应用程序实现如下功能: 第一个线程不停地随机生成[0,1)之间的浮点数 (float) 并输出到屏幕, 第二个线程将第一个线程输出的第 1-5 个浮点数的和与平均值输出到屏幕 (紧跟在第一个线程输出的第 5 个浮点数之后)、将第一个线程输出的第 6-10 个[0,1)之间的浮点数的和与平均值输出到屏幕 (紧跟在第一个线程输出的第 10 个浮点数之后) ...。要求线程间实现通信。要求采用实现 Runnable 接口和 Thread 类的构造方法的方式创建线程, 而不是通过 Thread 类的子类的方式。在报告中附上程序截图、运

行结果截图和详细的文字说明（包括设计的思路 and 合理性分析）。（10 分）

(3.2). 编写 Java 应用程序实现如下功能：创建工作线程，模拟银行现金账户取款和存款操作。多个线程同时执行取款和存款操作时，如果不使用同步处理，会造成账户余额混乱，要求使用 `synchronized` 关键字同步代码块，以保证多个线程同时执行取款和存款操作时，银行现金账户取款和存款的有效和一致。要求采用实现 `Runnable` 接口和 `Thread` 类的构造方法的方式创建线程，而不是通过 `Thread` 类的子类的方式。在报告中附上程序截图（假设银行存款有 100 元，有 3 个取款线程和 2 个存款线程，每次取款和存款均为 10 元）、运行结果截图（显示每次存取款操作后的余额等信息，以说明线程间同步正确）和详细的文字说明。（10 分）

(3.3). 有一座南北向的桥，只能容纳一个人，桥的南边有 1000 个人（记为 $S_1, S_2, \dots, S_{1000}$ ）和桥的北边有 1000 个人（记为 $N_1, N_2, \dots, N_{1000}$ ），编写 Java 应用程序让这些到达对岸，每个人用一个线程表示，桥为共享资源，在过桥的过程中输出谁正在过桥（不同人之间用逗号隔开）。运行 10 次，分别统计南边的 1000 人和北边的 1000 人先全部到达对岸的次数（第 i 行输出格式为：第 i 次运行，南边/北边先完成过桥）。要求采用实现 `Runnable` 接口和 `Thread` 类的构造方法的方式创建线程，而不是通过 `Thread` 类的子类的方式。在报告中附上程序截图、运行结果截图和详细的文字说明（包括对结果的分析）。（10 分）

报告写作。要求：主要思路有明确的说明，重点代码有详细的注释，行文逻辑清晰可读性强，报告整体写作较为专业。（20 分）

说明：

- (1) 本次实验课作业满分为 100 分，占总成绩的比例 7%。
- (2) 本次实验课作业截至时间 2024 年 11 月 20 日（周三）21:59。
- (3) 报告正文：请在**指定位置填写**，本次实验**不需要单独提交源程序文件**。
- (4) 个人信息：WORD 文件名中的“姓名”、“学号”，请改为你的**姓名和学号**；实验报告的首页，**请准确填写“学院”、“专业”、“报告人”、“学号”、“班级”、“实验报告提交时间”**等信息。
- (5) 提交方式：截至时间前，请在 Blackboard 平台中提交。
- (6) 发现抄袭（包括复制&粘贴整句话、整张图），**抄袭者和被抄袭者的成绩记零分**。
- (7) 延迟提交，不得分；如有特殊情况，请于截止日期之后的 **48 小时内**发邮件到 panweike@szu.edu.cn，并在邮件中注明课程名称、作业名称、姓名、学号等信息，以及特殊情况的说明，我收到后会及时回复。
- (8) 期末考试阶段补交无效。

Part 1 (25 分)

(1.1). 编写 Java 应用程序，实现浮点数（float）稀疏矩阵的乘法和加法运算，其中稀疏矩阵是指矩阵中的绝大部分元素的值为 0。在命令行读入和输出矩阵中的元素的时候，采用三元组的方式，即行号、列号和数值，例如“第 2 行、第 3 列、数值 3.2”表示为 2 3 3.2。要求以 1 个 3*4 的矩阵 ([1.2 0 0 0; 0 0 3.1 0; 0 0 0 2.2]) 和一个 4*5 的矩阵 ([0 1.1 1.5 0 0; 0 0 3.2 0 0; 0 1.3 0 0 -3.2; -1.0 6.2 0 0 0]) 相乘，以及 1 个 3*4 的矩阵 ([1.2 0 0 0; 0 0 3.1 0; 0 0 0 2.2]) 和 1 个 3*4 的矩阵 ([1.2 -3 -5.3 0; 0 1 0.1 -0.4; 2 2 1 0.2]) 相加为例，在报告中附上程序截图、完整的运行结果截图和简要文字说明。（10 分）

程序说明：

题目当中需要进行乘法和加法运算的矩阵为稀疏矩阵，故考虑使用 HashMap 库类进行处理。编写稀疏矩阵类，通过构造函数，由于输入的非零元素较少，故编写添加元素方法接受矩阵的数据，编写加、乘方法与打印方法。然后在主类中调用类中方法使用题中数据进行初始化与加乘运算后，最后输出结果。

程序内容：

```
1 package com.chino;
2
3 import java.util.*;
4
5 // 稀疏矩阵类
6 public class SparseMatrix {
7
8     // 使用HashMap存储稀疏矩阵的非零元素
9     private HashMap<Integer, HashMap<Integer, Float>> matrix;
10    private int rows, cols; // 矩阵的行数和列数
11
12    // 构造函数，初始化矩阵的行数和列数
13    public SparseMatrix(int rows, int cols) {
14        this.rows = rows;
15        this.cols = cols;
16        this.matrix = new HashMap<>();
17    }
18
19    // 添加非零元素到矩阵中
20    public void addValue(int row, int col, float value) {
21        if (value != 0) { // 仅存储非零元素
22            matrix.computeIfAbsent(row, k -> new HashMap<>()).put(col, value);
23        }
24    }
25
26    // 矩阵加法方法
27    public SparseMatrix add(SparseMatrix other) {
28        // 确保两个矩阵的维度一致
29        if (this.rows != other.rows || this.cols != other.cols) {
30            throw new IllegalArgumentException(s:"矩阵维度不匹配，无法进行加法运算");
31        }
32
33        SparseMatrix result = new SparseMatrix(this.rows, this.cols);
34
35        // 遍历当前矩阵，将值添加到结果矩阵
36        for (int row : this.matrix.keySet()) {
37            for (int col : this.matrix.get(row).keySet()) {
38                result.addValue(row, col, this.matrix.get(row).get(col));
39            }
40        }
41
42        // 遍历另一个矩阵，将值添加到结果矩阵
43        for (int row : other.matrix.keySet()) {
44            for (int col : other.matrix.get(row).keySet()) {
45                result.addValue(row, col, result.matrix.getOrDefault(row, new HashMap<>()).getOrDefault(col, defaultValue:0f) + other.matrix.get(row).get(col));
46            }
47        }
48
49        return result;
50    }
51
52    // 矩阵乘法方法
53    public SparseMatrix multiply(SparseMatrix other) {
54        // 确保当前矩阵的列数等于另一个矩阵的行数
55        if (this.cols != other.rows) {
56            System.out.println(x:"矩阵维度不匹配，无法进行乘法运算");
57        }
58
59        SparseMatrix result = new SparseMatrix(this.rows, other.cols);
60
61        // 遍历当前矩阵的所有非零元素
62        for (int row : this.matrix.keySet()) {
63            for (int col : this.matrix.get(row).keySet()) {
64                float valueA = this.matrix.get(row).get(col);
65
66                // 如果另一矩阵在对应位置有值
67                if (other.matrix.containsKey(col)) {
68                    for (int otherCol : other.matrix.get(col).keySet()) {
69                        float valueB = other.matrix.get(col).get(otherCol);
70                        // 结果矩阵元素累加
71                        result.addValue(row, otherCol, result.matrix.getOrDefault(row, new HashMap<>()).getOrDefault(otherCol, defaultValue:0f) + valueA * valueB);
72                    }
73                }
74            }
75        }
76
77        return result;
78    }
79 }
```

图一 1.1 程序内容①

```

80 // 打印矩阵的非零元素，以三元组的形式输出
81 public void print() {
82     for (int row : matrix.keySet()) {
83         for (int col : matrix.get(row).keySet()) {
84             System.out.printf(format: "%d %d %.2f\n", row, col, matrix.get(row).get(col));
85         }
86     }
87 }
88
89 Run [Debug] 运行 main | 调试 main
90 public static void main(String[] args) {
91     // 初始化第一个矩阵
92     SparseMatrix matrix1 = new SparseMatrix(rows:3, cols:4);
93     matrix1.addValue(row:0, col:0, value:1.2f);
94     matrix1.addValue(row:1, col:2, value:3.1f);
95     matrix1.addValue(row:2, col:3, value:2.2f);
96
97     // 初始化第二个矩阵
98     SparseMatrix matrix2 = new SparseMatrix(rows:4, cols:5);
99     matrix2.addValue(row:0, col:1, value:1.1f);
100    matrix2.addValue(row:0, col:2, value:1.5f);
101    matrix2.addValue(row:2, col:2, value:3.2f);
102    matrix2.addValue(row:2, col:4, -3.2f);
103    matrix2.addValue(row:3, col:0, -1.0f);
104    matrix2.addValue(row:3, col:1, value:6.2f);
105
106    // 矩阵乘法
107    System.out.println(x:"矩阵乘法结果: ");
108    SparseMatrix resultMul = matrix1.multiply(matrix2);
109    resultMul.print();
110
111    // 初始化第三个矩阵
112    SparseMatrix matrix3 = new SparseMatrix(rows:3, cols:4);
113    matrix3.addValue(row:0, col:0, value:1.2f);
114    matrix3.addValue(row:0, col:1, -3f);
115    matrix3.addValue(row:0, col:2, -5.3f);
116    matrix3.addValue(row:1, col:1, value:1f);
117    matrix3.addValue(row:1, col:2, value:0.1f);
118    matrix3.addValue(row:1, col:3, -0.4f);
119    matrix3.addValue(row:2, col:0, value:2f);
120    matrix3.addValue(row:2, col:1, value:2f);
121    matrix3.addValue(row:2, col:2, value:1f);
122    matrix3.addValue(row:2, col:3, value:0.2f);
123
124    // 矩阵加法
125    System.out.println(x:"\n矩阵加法结果:");
126    SparseMatrix resultAdd = matrix1.add(matrix3);
127    resultAdd.print();
128 }
129 }
130

```

图二 1.1 程序内容②

输出结果:

矩阵乘法结果:

```

0 1 1.32
0 2 1.80
1 2 9.92
1 4 -9.92
2 0 -2.20
2 1 13.64

```

矩阵加法结果:

```

0 0 2.40
0 1 -3.00
0 2 -5.30
1 1 1.00
1 2 3.20
1 3 -0.40
2 0 2.00
2 1 2.00
2 2 1.00
2 3 2.40

```

图三 1.1 程序输出

(1.2). 有 12 个国家（美国、中华人民共和国、德国、日本、英国、印度、法国、意大利、加拿大、韩国、以色列、俄罗斯），其属性有 name、GDP2023 和 Olympics2024，分别表示国家名称、世界银行公布的 2023 年的国内生产总值（单位：百万美元）和在 2024 年巴黎奥运会上获得的奖牌数量。

编写一个 Java 应用程序，要求使用 TreeSet。（i）按照 Olympics2024 从大到小排序输出这些国家的信息；（ii）按照 GDP2023 从大到小排序输出这些国家的信息。要求以上（i）和（ii）两小题都通过以下两种方式实现：通过实现 Comparator 接口或通过实现 Comparable 接口。在报告中附上程序截图、完整的运行结果截图和详细的文字说明。（15 分）

程序说明：

编写 Country 类表示一个国家，分别定义三个属性：name 表示国家名称，GDP2023 表示 2023 年的 GDP（以百万美元为单位），Olympics2024 表示 2024 年奥运会奖牌数量。通过构造函数初始化这些属性，并提供了相应的 getter 方法。为了实现排序，Country 类实现了 Comparable<Country> 接口，并重写了 compareTo 方法，按照 Olympics2024 从大到小排序。此外，还定义了两个静态内部类 GDPComparator 和 OlympicsComparator，分别实现 Comparator<Country> 接口，用于按 GDP2023 和 Olympics2024 从大到小排序。toString 方法被重写，用于格式化输出国家名称、GDP 和奥运会奖牌数量。在 Main 类的 main 方法中，首先创建了两个 TreeSet，分别用于按奥运会奖牌数量和 GDP 排序。然后，创建一个包含多个国家对象的列表，每个对象包含国家名称、2023 年的 GDP 和 2024 年奥运会奖牌数量。接着，将国家对象添加到 TreeSet 中，并按不同的排序方式输出结果。最后编写主类负责创建国家对象并将收集到的输入传入变量当中，最后对其进行排序和输出。

程序：

```
1 package com.chino;
2
3 import java.text.DecimalFormat;
4 import java.util.*;
5
6 class Country implements Comparable<Country> {
7     private String name;
8     private double GDP2023;
9     private int Olympics2024;
10
11     // 构造函数，初始化国家名称、GDP和奥运会奖牌的数量
12     public Country(String name, double GDP2023, int Olympics2024) {
13         this.name = name;
14         this.GDP2023 = GDP2023;
15         this.Olympics2024 = Olympics2024;
16     }
17
18     // 获取国家名称
19     public String getName() {
20         return name;
21     }
22
23     // 获取GDP
24     public double getGDP2023() {
25         return GDP2023;
26     }
27
28     // 获取奥运会的数量
29     public int getOlympics2024() {
30         return Olympics2024;
31     }
32
33     // 1. 按照Olympics2024从大到小排序
34     @Override
35     public int compareTo(Country o) {
36         return Integer.compare(o.Olympics2024, this.Olympics2024);
37     }
38
39     @Override
40     public String toString() {
41         DecimalFormat df = new DecimalFormat(pattern: "#,###");
42         return name + ", GDP2023: " + df.format(GDP2023) + "百万美元, Olympics2024: " + Olympics2024;
43     }
44 }
```

图四 1.2 程序内容①

```

45 // 2. Comparator实现: 按GDP2023从大到小排序
46 public static class GDPComparator implements Comparator<Country> {
47     @Override
48     public int compare(Country c1, Country c2) {
49         return Double.compare(c2.getGDP2023(), c1.getGDP2023());
50     }
51 }
52
53 // 3. Comparator实现: 按Olympics2024从大到小排序
54 public static class OlympicsComparator implements Comparator<Country> {
55     @Override
56     public int compare(Country c1, Country c2) {
57         return Integer.compare(c2.getOlympics2024(), c1.getOlympics2024());
58     }
59 }
60 }
61
62 public class Main {
63     Run | Debug | 运行 main | 调试 main.
64     public static void main(String[] args) {
65         // 创建按奥运会奖牌数量排序的TreeSet
66         TreeSet<Country> setByOlympics = new TreeSet<>(new Country.OlympicsComparator());
67         // 创建按GDP排序的TreeSet
68         TreeSet<Country> setByGDP = new TreeSet<>(new Country.GDPComparator());
69
70         // 数据输入
71         List<Country> countries = Arrays.asList(
72             new Country(name:"美国", GDP2023:27361000, Olympics2024:126),
73             new Country(name:"中华人民共和国", GDP2023:17795000, Olympics2024:91),
74             new Country(name:"德国", GDP2023:4456000, Olympics2024:33),
75             new Country(name:"日本", GDP2023:4213000, Olympics2024:45),
76             new Country(name:"印度", GDP2023:3550000, Olympics2024:6),
77             new Country(name:"英国", GDP2023:3340000, Olympics2024:65),
78             new Country(name:"法国", GDP2023:3031000, Olympics2024:64),
79             new Country(name:"意大利", GDP2023:2255000, Olympics2024:40),
80             new Country(name:"加拿大", GDP2023:2140000, Olympics2024:27),
81             new Country(name:"韩国", GDP2023:1713000, Olympics2024:32),
82             new Country(name:"以色列", GDP2023:5100000, Olympics2024:7),
83             new Country(name:"俄罗斯", GDP2023:2021000, Olympics2024:0)
84         );
85
86         // (i) 按Olympics2024排序
87         setByOlympics.addAll(countries);
88         System.out.println(x:"Olympics2024排序 (从大到小):");
89         for (Country country : setByOlympics) {
90             System.out.println(country);
91         }
92
93         // (ii) 按GDP2023排序
94         setByGDP.addAll(countries);
95         System.out.println(x:"\nGDP2023排序 (从大到小):");
96         for (Country country : setByGDP) {
97             System.out.println(country);
98         }
99     }
100 }

```

图五 1.2 程序内容②

程序输出:

```

Olympics2024排序 (从大到小):
美国, GDP2023: 27,361,000百万美元, Olympics2024: 126
中华人民共和国, GDP2023: 17,795,000百万美元, Olympics2024: 91
英国, GDP2023: 3,340,000百万美元, Olympics2024: 65
法国, GDP2023: 3,031,000百万美元, Olympics2024: 64
日本, GDP2023: 4,213,000百万美元, Olympics2024: 45
意大利, GDP2023: 2,255,000百万美元, Olympics2024: 40
德国, GDP2023: 4,456,000百万美元, Olympics2024: 33
韩国, GDP2023: 1,713,000百万美元, Olympics2024: 32
加拿大, GDP2023: 2,140,000百万美元, Olympics2024: 27
以色列, GDP2023: 5,100,000百万美元, Olympics2024: 7
印度, GDP2023: 3,550,000百万美元, Olympics2024: 6
俄罗斯, GDP2023: 2,021,000百万美元, Olympics2024: 0

GDP2023排序 (从大到小):
美国, GDP2023: 27,361,000百万美元, Olympics2024: 126
中华人民共和国, GDP2023: 17,795,000百万美元, Olympics2024: 91
以色列, GDP2023: 5,100,000百万美元, Olympics2024: 7
德国, GDP2023: 4,456,000百万美元, Olympics2024: 33
日本, GDP2023: 4,213,000百万美元, Olympics2024: 45
印度, GDP2023: 3,550,000百万美元, Olympics2024: 6
英国, GDP2023: 3,340,000百万美元, Olympics2024: 65
法国, GDP2023: 3,031,000百万美元, Olympics2024: 64
意大利, GDP2023: 2,255,000百万美元, Olympics2024: 40
加拿大, GDP2023: 2,140,000百万美元, Olympics2024: 27
俄罗斯, GDP2023: 2,021,000百万美元, Olympics2024: 0
韩国, GDP2023: 1,713,000百万美元, Olympics2024: 32

```

图六 1.2 程序输出

Part 2 (25 分)

(2.1) 将第 8 章讲义 (JavaPD-Ch08) 中的 5 个应用程序 (Example8_1, Example8_2, Example8_3, Example8_4, Example8_6) 在 Eclipse 中运行, 如运行结果不唯一, 则需要运行多次并至少得到两个不同的结果。对重要语句加上注释。在报告中附上程序截图、运行结果截图和简要文字说明 (对运行结果做出解释)。(5 分)

程序与运行结果:

Example8_1:

```
1  package com.chino;
2
3  // 定义一个继承自Thread的类WriteWordThread
4  class WriteWordThread extends Thread {
5      // 构造方法, 传入一个字符串参数, 用于设置线程名称
6      WriteWordThread(String s) {
7          setName(s);
8      }
9
10     // 重写run方法, 定义线程要执行的任务
11     public void run() {
12         for (int i = 0; i < 10; i++) {
13             System.out.println("Thread: "+getName());
14         }
15     }
16 }
17
18 // 主类Example8_1
19 public class Example8_1{
20     // 主方法
21     Run | Debug | 运行 main | 调试 main,
22     public static void main(String[] args) {
23         // 定义两个WriteWordThread对象
24         WriteWordThread zhang,wang;
25         // 创建zhang对象, 并设置线程名称为"Zhang"
26         zhang = new WriteWordThread(s:"Zhang");
27         // 创建wang对象, 并设置线程名称为"Wang"
28         wang = new WriteWordThread(s:"Wang");
29         // 启动zhang线程
30         zhang.start();
31         // 主线程输出3次"Main Thread"
32         for(int i=1;i<=3;i++){
33             System.out.println(x:"Main Thread");
34         }
35         // 启动wang线程
36         wang.start();
37     }
38 }
```

图七 2.1 Example8_1 程序

Example8_1 运行结果:

Main Thread	Main Thread
Main Thread	Main Thread
Main Thread	Main Thread
Thread: Zhang	Thread: Wang
Thread: Zhang	Thread: Wang
Thread: Zhang	Thread: Wang
Thread: Zhang	Thread: Wang
Thread: Zhang	Thread: Wang
Thread: Wang	Thread: Zhang
Thread: Zhang	Thread: Zhang
Thread: Wang	Thread: Wang
Thread: Wang	Thread: Wang
Thread: Wang	Thread: Wang
Thread: Wang	Thread: Wang
Thread: Wang	Thread: Wang
Thread: Zhang	Thread: Zhang
Thread: Wang	Thread: Zhang
Thread: Wang	Thread: Zhang
Thread: Wang	Thread: Zhang
Thread: Wang	Thread: Zhang
Thread: Wang	Thread: Zhang
Thread: Zhang	Thread: Zhang
Thread: Zhang	Thread: Zhang
Thread: Zhang	Thread: Wang

图八、九 2.1 Example8_1 程序输出①②

Example8_2:

```
1  package com.chino;
2
3  // 定义一个继承自Thread的类WriteWordThread
4  class WriteWordThread extends Thread {
5      int n = 0; // 线程休眠时间
6
7      // 构造方法, 接收线程名称和休眠时间
8      public WriteWordThread(String s, int n) {
9          setName(s); // 设置线程名称
10         this.n = n; // 设置休眠时间
11     }
12
13
14     // 重写run方法, 定义线程执行的任务
15     public void run() {
16         for (int i = 1; i <= 3; i++) { // 循环3次
17             System.out.println("Thread: " + getName()); // 输出线程名称
18             try {
19                 sleep(n); // 线程休眠指定时间
20             } catch (InterruptedException e) {
21                 // 捕获并忽略中断异常
22             }
23         }
24     }
25 }
26
27
28 public class Example8_2 {
29     Run | Debug | 运行 main | 调试 main,
30     public static void main(String args[]) {
31         WriteWordThread zhang, wang; // 声明两个WriteWordThread对象
32         zhang = new WriteWordThread(s:"Zhang", n:200); // 创建线程对象zhang, 休眠时间200毫秒
33         wang = new WriteWordThread(s:"Wang", n:100); // 创建线程对象wang, 休眠时间100毫秒
34         zhang.start(); // 启动线程zhang
35         wang.start(); // 启动线程wang
36     }
37 }
```

图十 2.1 Example8_2 程序

输出结果:

Thread: Wang	Thread: Wang
Thread: Zhang	Thread: Zhang
Thread: Wang	Thread: Wang
Thread: Wang	Thread: Wang
Thread: Zhang	Thread: Zhang
Thread: Zhang	Thread: Zhang

图十一、十二 2.1 Example8_2 程序输出①②

Example8_3:

```
1 package com.chino;
2
3 // 定义一个Left类, 继承自Thread类
4 class Left extends Thread {
5     int n = 0; // 计数器
6
7     // 重写run方法
8     public void run() {
9         while (true) {
10             n++;
11             System.out.println(n + "Left");
12             try {
13                 // 随机休眠一段时间
14                 sleep((int) (Math.random() * 100));
15             } catch (InterruptedException e) {
16                 // 捕获并忽略中断异常
17             }
18         }
19     }
20 }
21
22 // 定义一个Right类, 继承自Thread类
23 class Right extends Thread {
24     int n = 0; // 计数器
25
26     // 重写run方法
27     public void run() {
28         while (true) {
29             n++;
30             System.out.println(n + "Right");
31             try {
32                 // 随机休眠一段时间
33                 sleep((int) (Math.random() * 100));
34             } catch (InterruptedException e) {
35                 // 捕获并忽略中断异常
36             }
37         }
38     }
39 }
40
41 // 定义一个Example8_3类
42 public class Example8_3 {
43     // Run | Debug | 运行 main | 调试 main
44     public static void main(String args[]) {
45         // 创建Left和Right对象
46         Left left = new Left();
47         Right right = new Right();
48         // 启动Left和Right线程
49         left.start();
50         right.start();
51         while (true) {
52             try {
53                 // 主线程休眠100毫秒
54                 Thread.sleep(100);
55             } catch (InterruptedException e) {
56                 e.printStackTrace();
57             }
58             // 如果Left或Right的计数器大于等于8, 则输出计数器的值并退出程序
59             if (left.n >= 8 || right.n >= 8) {
60                 System.out.println(left.n + "," + right.n);
61                 System.exit(status:0);
62             }
63         }
64     }
65 }
```

图十三 2.1 Example8_3 程序

输出结果:

1Left	1Left
1Right	4Left
2Right	3Right
2Left	5Left
3Left	4Right
3Right	5Right
4Left	6Left
4Right	6Right
5Right	7Right
5Left	7Left
6Left	8Left
6Right	9Left
7Right	10Left
7Left	11Left
8Right	12Left
9Right	8Right
8Left	11,7
10Right	9Right
8,9	
9Left	

图十四、十五 2.1 Example8_3 程序输出①②

Example8_4:

```

1 package com.chino;
2
3 class TaskBank implements Runnable {
4     private int money = 0; // 账户余额
5     String name1, name2; // 线程名称
6
7     // 构造方法, 初始化线程名称
8     TaskBank(String s1, String s2) {
9         name1 = s1;
10        name2 = s2;
11    }
12
13    // 设置账户余额
14    public void setMoney(int mount) {
15        money = mount;
16    }
17
18    public void run() {
19        while (true) {
20            money = money - 10;
21            // 判断当前线程名称, 输出账户余额
22            if (Thread.currentThread().getName().equals(name1)) {
23                System.out.println(name1 + ": " + money);
24                // 如果账户余额小于等于100, 输出完成信息并结束线程
25                if (money <= 100) {
26                    System.out.println(name1 + ": Finished");
27                    return;
28                }
29            } else if (Thread.currentThread().getName().equals(name2)) {
30                System.out.println(name2 + ": " + money);
31                // 如果账户余额小于等于60, 输出完成信息并结束线程
32                if (money <= 60) {
33                    System.out.println(name2 + ": Finished");
34                    return;
35                }
36            }
37            try {
38                // 暂停线程800毫秒
39                Thread.sleep(800);
40            } catch (InterruptedException e) {
41                // 捕获并忽略中断异常
42            }
43        }
44    }
45 }
46
47 public class Example8_4 {
48     Run | Debug | 运行 main | 调试 main.
49     public static void main(String args[]) {
50         String s1 = "treasurer zhang"; // 会计
51         String s2 = "cashier cheng"; // 出纳
52         TaskBank taskBank = new TaskBank(s1, s2);
53         taskBank.setMoney(mount:120);
54         Thread zhang;
55         Thread cheng;
56         zhang = new Thread(taskBank); // 目标对象bank
57         cheng = new Thread(taskBank); // 目标对象bank
58         zhang.setName(s1);
59         cheng.setName(s2);
60         zhang.start();
61         cheng.start();
62     }
63 }

```

图十六 2.1 Example8_4 程序

输出结果:

treasurer zhang: 100	treasurer zhang: 110
cashier cheng: 100	cashier cheng: 100
treasurer zhang: Finished	treasurer zhang: Finished
cashier cheng: 90	cashier cheng: 90
cashier cheng: 80	cashier cheng: 80
cashier cheng: 70	cashier cheng: 70
cashier cheng: 60	cashier cheng: 60
cashier cheng: Finished	cashier cheng: Finished

图十七、十八 2.1 Example8_4 程序输出①②

Example8_6:

```

1  package com.chino;
2
3  class TaskAddSub implements Runnable {
4      String s1, s2; // 线程名称
5
6      TaskAddSub(String s1, String s2) {
7          this.s1 = s1;
8          this.s2 = s2;
9      }
10
11     public void run() {
12         // 定义一个变量i, 初始值为0
13         int i = 0;
14         // 无限循环
15         while (true) {
16             // 如果当前线程的名字等于s1
17             if (Thread.currentThread().getName().equals(s1)) {
18                 // i加1
19                 i = i + 1;
20                 // 输出s1:i
21                 System.out.println(s1 + ":" + i);
22                 // 如果i大于等于4
23                 if (i >= 4) {
24                     // 输出s1Finished
25                     System.out.println(s1 + "Finished");
26                     // 返回
27                     return;
28                 }
29             } // 如果当前线程的名字等于s2
30             else if (Thread.currentThread().getName().equals(s2)) {
31                 // i减1
32                 i = i - 1;
33                 // 输出s2:i
34                 System.out.println(s2 + ":" + i);
35                 if (i <= -4) {
36                     System.out.println(s2 + "Finished");
37                     return;
38                 }
39             }
40             try {
41                 Thread.sleep(millis:800);
42             } catch (InterruptedException e) {
43                 // 捕获并忽略中断异常
44             }
45         }
46     }
47 }
48
49 // 定义一个类Example8_6
50 public class Example8_6 {
51     // 主方法
52     public static void main(String args[]) {
53         // 定义两个字符串变量s1和s2
54         String s1 = "ZHANG San";
55         String s2 = "LI Si";
56         // 创建一个TaskAddSub对象, 传入s1和s2
57         TaskAddSub taskAddSub = new TaskAddSub(s1, s2);
58
59         // 定义两个Thread对象zhang和li
60         Thread zhang, li;
61         // 创建两个Thread对象, 传入taskAddSub
62         zhang = new Thread(taskAddSub);
63         li = new Thread(taskAddSub);
64         // 设置zhang和li的名称为s1和s2
65         zhang.setName(s1);
66         li.setName(s2);
67         // 启动zhang和li线程
68         zhang.start();
69         li.start();
70     }
71 }

```

图十九 2.1 Example8_6 程序输出①②

运行结果:

```

ZHANG San:1      ZHANG San:1
LI Si:-1         LI Si:-1
LI Si:-2         ZHANG San:2
ZHANG San:2      LI Si:-2
LI Si:-3         ZHANG San:3
ZHANG San:3      LI Si:-3
LI Si:-4         ZHANG San:4
ZHANG San:4      LI Si:-4
LI SiFinished    LI SiFinished
ZHANG SanFinished ZHANG SanFinished

```

图二十、二十一 2.1 Example8_6 程序输出①②

运行结果解释:

- (1) **Example8_1:** 运行结果当中包括 10 次 "Thread: Zhang"、3 次 "Main Thread"、10 次 "Thread: Wang"。但是每一次运行的结果当中, 上述三个输出在程序实际输出当中顺序不固定, 并且基本上每一次运行的结果都与上一次不同。
原因是每一次线程的执行顺序和时间分配都是由操作系统调度的, 其线程的执行顺序以及其运行后展现出的输出顺序会受到每一次运行后操作系统调度的影响
- (2) **Example8_2:** 运行结果当中包含 3 次 "Thread: Zhang"、3 次 "Thread: Wang"。但是每一次运行的结果当中, 上述三个输出在程序实际输出当中顺序不同, 但相对于 Example8_1, 在许多时候的输出都会于上一次相同。
原因是由于加入的延时的存在导致 Wang 线程的输出频率更高, 从而导致相对于 Example8_1, 其输出规律性增加了。
- (3) **Example8_3:** 运行结果中包含一个大于 7 次的"(n)left"或"(n)Right", 而另外一个则不会大于七次。每一次运行的结果当中上述输出的顺序不固定, 相对于 Example8_1 当中, 两个输出的输出个数在每一次运行当中也不固定, 并且波动较大。
原因是和 Example8_1 中一样每一次线程的执行顺序和时间分配都是由操作系统调度的, 其线程的执行顺序以及其运行后展现出的输出顺序会受到每一次运行后操作系统调度的影响, 但由于程序当中给出了 Left 或 Right 的计数器大于等于 8 的退出条件, 会导致某个线程运行到退出条件时, 会直接结束另外一个线程的运行, 由于调度的随机性, 该线程退出时的实际运行次数不固定, 故导致最后的输出次数波动较大。
- (4) **Example8_4:** 运行结果中包含"treasurer zhang: n" cashier cheng: m", 其中 m 会从 100 逐个的输出到 60, 但是 n 只会输出一钱 money=110 后便输出终止语句"treasurer zhang: Finished"。在最后才输出另外一个线程的终止语句"cashier cheng: Finished"。每一次运行的结果当中上述输出一开始的顺序不固定, 但在 treasurer 线程终止后稳定。同时 treasurer 没有输出临界条件 money=100, 但 cashier 输出了临界条件 money=60
- (5) **原因是:** ①两个线程一开始同时运行的时候会将 money 变量同时减去两次, 导致 treasurer 达到条件后直接退出, 导致这个变量没有对 100 的情况进行输出。②和上述情况一样, 每一次线程的执行顺序和时间分配都是由操作系统调度的, 其线程的执行顺序以及其运行后展现出的输出顺序会受到每一次运行后操作系统调度的影响, 在其中一个线程结束之后由于只剩下一个线程, 便不会受到影响。
- (6) **Example8_6:** 随机顺序输出"LI Si:i","ZHANG San:i", 与 Example8_4 不同的是, 前者的 i 从-1 输出到-4, 后者的 i 从 1 到 4。输出的顺序与 Example8_1 一样是乱序的。
原因是: ①i 变量的变化是在判断完为特定线程后的在其内部进行的, 这样会导致两个线程的 i 变量数据不互通, 导致其单独增加 i 或减少 i, 最后达到临界结束, 而二者并不会互相影响。②和上述情况一样, 每一次线程的执行顺序和时间分配都是由操作系统调度的, 其线程的执行顺序以及其运行后展现出的输出顺序会受到每一次运行后操作系统调度的影响, 故输出的顺序是乱的。

(2.2). 运行以下三个程序（每个程序运行 5 次），并对输出结果给出分析。在报告中附上程序截图和简要的文字说明（包括对结果的分析）。（10 分）

程序 1:

程序:

```
1 package com.chino;
2 class PrintChar implements Runnable{
3     private char charToPrint;
4     private int times;
5     public PrintChar(char c,int t){
6         this.charToPrint = c;
7         this.times = t;
8     }
9     @Override
10    public void run(){
11        for(int i=0;i<times;i++){
12            System.out.print(charToPrint);
13        }
14    }
15 }
16 class PrintNum implements Runnable{
17     private int lastNum;
18     public PrintNum(int n){
19         lastNum = n;
20     }
21     @Override
22     public void run(){
23         for(int i=1;i<=lastNum;i++){
24             System.out.print(" "+i);
25         }
26     }
27 }
28
29 public class TaskThreadDemo{
30     Run | Debug | 运行 main | 调试 main,
31     public static void main(String[] args) {
32         Runnable PrintA= new PrintChar(c:'a',t:100);
33         Runnable PrintB= new PrintChar(c:'b',t:100);
34         Runnable Print100= new PrintNum(n:100);
35
36         Thread thread1 = new Thread(PrintA);
37         Thread thread2 = new Thread(PrintB);
38         Thread thread3 = new Thread(Print100);
39
40         thread1.start();
41         thread2.start();
42         thread3.start();
43     }
44 }
```

图二十二 2.2 程序 1

输出结果:

```
abbbbbbbbbbaaaaaaaaaabbbbbbbbbbaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaabbbbbbaaaaaaaaaaaaaaaaaa
1 2baaaaaaaaaa 3 4 5 6 7 8 9b 10 11 12 13 14 15bbbbbbbb 16
bbbbbbbb 17 18bbbbbbbbbbbbbbbbbb 19bb 20bbbbbbbb 21bbbbbb
22bbbbbbbb 23bb 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38
39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58
59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 7
9 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99
100

aaaaaaaaaaaaaaaaabbbbbbaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbbbbbbaaaaaaaaaa
bbbbbbbbbbbbbbbb 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 1
8 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38
39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58
59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 7
9 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99
100
```

图二十三、二十四 2.2 程序 1 输出①②

结果总共输出了 100 个 a、100 个 b，以及从 1 到 100 的数字。先主要输出字母，然后再输出数字。字母快输入完毕时，会有数字与字母混杂在一起的情况。而前面的则是两个字母混杂在一起，并且是一长串的输出。

分析与原因：推测为线程调度当中字母打印的执行速度较快（其不用打印空格，并且打印的字符数较少等），而在后方是由于字母进程趋于结束，进程的消耗量减少，进而导致数字进程可以被系统执行。而在字母进程完全结束后，便是执行剩余的数字进程。

程序 2:

程序:

```
1 package com.chinoj;
2 import java.util.concurrent.*;
3
4 class PrintChar implements Runnable {
5     private char charToPrint; // 要打印的字符
6     private int times; // 打印次数
7
8     // 构造函数，初始化要打印的字符和打印次数
9     public PrintChar(char c, int t) {
10         this.charToPrint = c;
11         this.times = t;
12     }
13
14     // 实现Runnable接口的run方法，打印字符
15     @Override
16     public void run() {
17         for (int i = 0; i < times; i++) {
18             System.out.print(charToPrint);
19         }
20     }
21 }
22
23 class PrintNum implements Runnable {
24     private int lastNum; // 要打印的最大数字
25
26     // 构造函数，初始化要打印的最大数字
27     public PrintNum(int n) {
28         lastNum = n;
29     }
30
31     @Override
32     public void run() {
33         for (int i = 1; i <= lastNum; i++) {
34             System.out.print(" " + i);
35         }
36     }
37 }
38
39 public class ExecutorDemo {
40     Run | Debug | 运行 main | 调试 main
41     public static void main(String[] args) {
42         // 创建一个具有固定线程数的线程池
43         ExecutorService executor = Executors.newFixedThreadPool(nThreads:3);
44         // 执行打印字符'a'的任务
45         executor.execute(new PrintChar(c:'a', t:100));
46         // 执行打印字符'b'的任务
47         executor.execute(new PrintChar(c:'b', t:100));
48         // 执行打印数字的任务
49         executor.execute(new PrintNum(n:100));
50         // 关闭线程池
51         executor.shutdown();
52     }
53 }
```

图二十五 2.2 程序 2

输出结果:

```
aaaaaaaaaaaaaaaaabbbbbbbbbbbbaaaaaaaaaaaaaaaaaabbbbbbbbbbbbaabbb
bbbbbbbaaaaaaaaaabbbbbbbbaaaaaaaaaabbbbbbbbaaaaaaaaaaaaaaaaaa
abbbbbbbbbbbbbbbbbbbbaaaaaaaaaabbbbbbbbbbbbaaaaaaaaaabbb 3
4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51
52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74
75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97
98 99 100

aaaaaaaaaaaaaaaaabbbbbbbbaaaaaaaaaabbbbbbaaabbbbbbbbbbbbaaaaaa
aaaaabbbbbbaabbbbbbbbaaaaaaaaaabbbbbbbbaabbbbbbbbaabbbbaaaaa
aaabbbbbbbbaabbbbaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa 1 2 3
4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51
52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74
75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97
98 99 100
```

图二十六、二十七 2.2 程序 2 输出①②

和程序 1 一样输出了类似的 100 个 a、100 个 b，以及从 1 到 100 的数字。但是在输出当中如果数字穿插在字母之间，会有相对于程序 1 更大的间隔。并且数字穿插在字母当中的情况更少了。

分析与原因：在程序 1 的基础上，由于使用了 `ExecutorService` 进行进程的管理，导致输出时间较长的数字顺序被调至了更低的水平，体现在输出中就是穿插量的减少（也就是间隔增大）和有穿插情况的减少。

程序 3:

程序：

```
1 package com.chino;
2
3 import java.util.concurrent.*;
4 // 定义一个没有同步的账户类
5 public class AccountWithoutSync{
6     // 定义一个静态的账户对象
7     private static Account account = new Account();
8
9     Run | Debug | 运行 main | 调试 main,
10    public static void main(String[] args) throws InterruptedException {
11        // 创建一个线程池
12        ExecutorService executor = Executors.newCachedThreadPool();
13        // 循环100次，向线程池中添加任务
14        for (int i = 0; i < 100; i++) {
15            executor.execute(new AddApennyTask());
16        }
17        // 关闭线程池
18        executor.shutdown();
19        // 等待线程池中的任务执行完毕
20        while (!executor.isTerminated()) {
21        }
22        // 输出账户余额
23        System.out.println("What is balance? " + account.getBalance());
24    }
25    // 定义一个添加一分钱的任務类
26    private static class AddApennyTask implements Runnable {
27        public void run() {
28            // 调用账户对象的存款方法
29            account.deposit(amount:1);
30        }
31    }
32    // 定义一个账户类
33    private static class Account{
34        // 定义账户余额
35        private int balance = 0;
36
37        // 获取账户余额
38        public int getBalance(){
39            return balance;
40        }
41        // 存款方法
42        public void deposit(int amount){
43            // 定义新的余额
44            int newBalance = balance + amount;
45            try {
46                // 模拟耗时操作
47                Thread.sleep(millis:5);
48            } catch (InterruptedException ex) {
49            }
50            // 更新余额
51            balance = newBalance;
52        }
53    }
54 }
55
56 }
```

图二十八 2.2 程序 3

输出结果：

```
What is balance? 3
What is balance? 2
```

图二十九、三十 2.2 程序 3 输出①②

输出了最后的钱数及其提示，并且等待时间越长，输出的钱数越少，波动也较大

原因：由于 Account 类的 deposit 方法没有同步机制，多个线程可能会同时访问和修改 balance 变量，导致竞态条件（race condition）。因此，最终的账户余额可能不等于预期的 100，实际使用当中，等待时间越长会导致同一时间有更多的线程访问 balance 变量，导致最终的余额下降。反之亦然。

(2.3) 第 8 章讲义（JavaPD-Ch08）中的第 5 个应用程序（Example8_5）存在线程间不同步的问题，请修改该程序，以解决不同步的问题。在报告中附上程序截图、运行结果截图和详细的文字说明（包括设计的思路和合理性分析）。（10 分）

原程序：

```
1 package com.chino;
2
3 class Task implements Runnable {
4     private int number = 0; // 共享变量
5
6     // 设置number的值
7     public synchronized void setNumber(int n) {
8         number = n;
9     }
10
11     public void run() {
12         // 无限循环
13         while (true) {
14             // 同步代码块，保证线程安全
15             synchronized (this) {
16                 // 如果当前线程的名字是"add", 则number加1, 并输出
17                 if (Thread.currentThread().getName().equals("add")) {
18                     number++;
19                     System.out.printf(format: "%d\n", number);
20                 }
21                 // 如果当前线程的名字是"sub", 则number减1, 并输出
22                 if (Thread.currentThread().getName().equals("sub")) {
23                     number--;
24                     System.out.printf(format: "%d\n", number);
25                 }
26             }
27             try {
28                 Thread.sleep(millis:1000);
29             } catch (InterruptedException e) {
30                 // 捕获并忽略中断异常
31             }
32         }
33     }
34 }
35
36 public class Example8_5 {
37     run | Debug | 运行 main | 调试 main
38     public static void main(String args[]) {
39         // 创建一个Task对象, 并设置其number属性为10
40         Task taskAdd = new Task();
41         taskAdd.setNumber(n:10);
42         // 创建一个Task对象, 并设置其number属性为-10
43         Task taskSub = new Task();
44         taskSub.setNumber(-10);
45         // 创建四个线程对象
46         Thread threadA, threadB, threadC, threadD;
47         // 将taskAdd对象作为参数传入线程对象
48         threadA = new Thread(taskAdd);
49         threadB = new Thread(taskAdd);
50         // 设置线程A和线程B的名称为"add"
51         threadA.setName(name:"add");
52         threadB.setName(name:"add");
53         // 将taskSub对象作为参数传入线程对象
54         threadC = new Thread(taskSub);
55         threadD = new Thread(taskSub);
56         // 设置线程C和线程D的名称为"sub"
57         threadC.setName(name:"sub");
58         threadD.setName(name:"sub");
59         // 启动四个线程
60         threadA.start();
61         threadB.start();
62         threadC.start();
63         threadD.start();
64     }
65 }
```

图三十一 2.3 原程序

原输出：

```
16
17
18
19
20
-17
-18
-19
-20
```

图三十二 2.3 原程序输出

分析输出：观察程序 Example8_5 原输出发现其输出的两个变量是各自增加与减少的，没有对同一个变量进行修改。类似于 Example8_6 的情况。

修改思路：根据分析，我们需要确保所有的进程同步 `number`，在此之上，为了避免同时修改带来的错位，需要限制每一次只有一个进程访问 `number`，也就是进程安全。

修改方法：将所有线程共享同一个 `Task` 对象，以确保所有线程对 `number` 的访问是同步的。然后在 `run` 方法中使用同步代码块，编写设置和获取 `number` 的方法 `setNumber` 和 `getNumber`，并同时使用 `synchronized` 关键字，确保对 `number` 的访问是线程安全的。这样通过共享同一个 `Task` 对象，并在 `run` 方法中使用同步代码块，确保了对共享变量 `number` 的访问是线程安全的。这样可以避免多个线程同时修改 `number` 变量，导致数据不一致的问题。

修改后程序：

```
1 package com.chino;
2
3 class Task implements Runnable {
4     private int number = 0; // 共享变量
5
6     // 设置number的值
7     public synchronized void setNumber(int n) {
8         number = n;
9     }
10
11     // 获取number的值
12     public synchronized int getNumber() {
13         return number;
14     }
15
16     public void run() {
17         // 无限循环
18         while (true) {
19             // 同步代码块，保证线程安全
20             synchronized (this) {
21                 // 如果当前线程的名字是"add"，则number加1，并输出
22                 if (Thread.currentThread().getName().equals("add")) {
23                     number++;
24                     System.out.printf("add\n", number);
25                 }
26                 // 如果当前线程的名字是"sub"，则number减1，并输出
27                 if (Thread.currentThread().getName().equals("sub")) {
28                     number--;
29                     System.out.printf("sub\n", number);
30                 }
31             }
32             try {
33                 Thread.sleep(1000);
34             } catch (InterruptedException e) {
35                 // 捕获并忽略中断异常
36             }
37         }
38     }
39 }
40
41 public class Example8_5_change {
42     // Run | Debug | 运行 main | 调试 main
43     public static void main(String args[]) {
44         // 创建一个Task对象，并设置其number属性为0
45         Task task = new Task();
46         task.setNumber(0);
47
48         // 创建四个线程对象
49         Thread threadA, threadB, threadC, threadD;
50         // 将task对象作为参数传入线程对象
51         threadA = new Thread(task);
52         threadB = new Thread(task);
53         threadC = new Thread(task);
54         threadD = new Thread(task);
55
56         // 设置线程A和线程B的名称为"add"
57         threadA.setName("add");
58         threadB.setName("add");
59         // 设置线程C和线程D的名称为"sub"
60         threadC.setName("sub");
61         threadD.setName("sub");
62
63         // 启动四个线程
64         threadA.start();
65         threadB.start();
66         threadC.start();
67         threadD.start();
68     }
69 }
```

图三十三 2.3 修改后程序

最终结果：

```
0
-1
0
1
0
-1
0
1
0
```

图三十四 2.3 修改后程序输出

最后输出的数字是在某个数字附近摆动（这个数字取决于一开始的线程情况）。

Part 3 (30 分)

(3.1). 编写 Java 应用程序实现如下功能：第一个线程不停地随机生成[0,1)之间的浮点数（float）并输出到屏幕，第二个线程将第一个线程输出的第 1-5 个浮点数的和与平均值输出到屏幕（紧跟在第一个线程输出的第 5 个浮点数之后）、将第一个线程输出的第 6-10 个[0,1)之间的浮点数的和与平均值输出到屏幕（紧跟在第一个线程输出的第 10 个浮点数之后）…。要求线程间实现通信。要求采用实现 Runnable 接口和 Thread 类的构造方法的方式创建线程，而不是通过 Thread 类的子类的方式。在报告中附上程序截图、运行结果截图和详细的文字说明（包括设计的思路 and 合理性分析）。（10 分）

设计思路：按照题目的意思，需要设计两个线程，其中第一个断生成[0, 1)范围内的随机浮点，将其存储到共享列表 floatList 中并输出到屏幕上。然后在生成五个随机数的时候线程 1 通知线程 2 进行计算后输出，并进入等待状态，暂停生成操作。

线程 2 每次接到通知后，它计算最近生成的 5 个浮点数的和与平均值。计算完成后，输出结果并通知线程 1 继续生成新的浮点数，然后再次进入等待状态。直到接受到线程 1 的通知。显然，应该让线程 2 在启动时处于等待状态，等待线程 1 的通知。

合理性分析：

线程间通信：使用 synchronized 关键字和锁对象 lock 实现线程间的同步，确保生产者和消费者线程安全地访问共享列表。同时保证在生成足够数目随机数后将两个线程正确的停止。使用 wait 和 notify 方法实现线程间的通信，确保线程 1 在有足够数据时被唤醒进行处理，线程 2 在数据处理完毕后继续生成数据。

线程安全：通过同步代码块，确保对共享列表的访问是线程安全的，避免竞态条件。使用 wait 和 notify 方法确保线程在适当的时机被唤醒和等待，避免多个线程同时修改变量，导致数据不一致的问题。

程序：

```
1 package com.chino;
2
3 import java.util.ArrayList;
4 import java.util.Random;
5
6 public class FloatGenerator {
7     Run | Debug | 运行 main | 调试 main
8     public static void main(String[] args) {
9         ArrayList<Float> floatList = new ArrayList<>(); // 创建一个Float类型的ArrayList
10         Object lock = new Object(); // 共享锁对象用于线程通信
11
12         // 线程1: 生成浮点数
13         Runnable generatorTask = new Runnable() {
14             public void run() {
15                 Random random = new Random();
16                 while (true) {
17                     synchronized (lock) {
18                         float num = random.nextFloat(); // 生成随机浮点数
19                         floatList.add(num);
20                         System.out.println("Generated: " + num);
21
22                         // 生成5个数后通知计算线程
23                         if (floatList.size() % 5 == 0) {
24                             lock.notify(); // 通知计算线程
25                             try {
26                                 lock.wait(); // 生成线程等待
27                             } catch (InterruptedException e) {
28                                 e.printStackTrace();
29                             }
30                         }
31                     }
32
33                     // 限制生成速度
34                     try {
35                         Thread.sleep(millis:300); // 暂停300ms
36                     } catch (InterruptedException e) {
37                         e.printStackTrace();
38                     }
39                 }
40             }
41         };
42     }
43 }
```

图三十五 3.1 程序①

```

42 // 线程2: 计算和与平均值
43 Runnable calculatorTask = new Runnable() {
44     public void run() {
45         while (true) {
46             synchronized (lock) {
47                 try {
48                     // 等待生成线程通知
49                     lock.wait();
50
51                     // 计算最新5个浮点数的和与平均值
52                     int size = floatList.size();
53                     float sum = 0;
54                     for (int i = size - 5; i < size; i++) {
55                         sum += floatList.get(i);
56                     }
57                     float average = sum / 5;
58                     System.out.println("Sum: " + sum + ", Average: " + average);
59
60                     // 通知生成线程继续生成
61                     lock.notify();
62                 } catch (InterruptedException e) {
63                     e.printStackTrace();
64                 }
65             }
66         }
67     }
68 };
69
70 // 启动线程
71 new Thread(generatorTask).start(); // 启动生成线程
72 new Thread(calculatorTask).start(); // 启动计算线程
73 }
74 }

```

图三十六 3.1 程序②

程序输出:

```

Generated: 0.42555428
Generated: 0.97887343
Generated: 0.23372537
Generated: 0.5558527
Generated: 0.8779772
Sum: 3.0719829, Average: 0.6143966
Generated: 0.9723526
Generated: 0.71108526
Generated: 0.2934993
Generated: 0.37528062
Generated: 0.24236816
Sum: 2.594586, Average: 0.5189172
Generated: 0.10420811
Generated: 0.6004512
Generated: 0.31202316
Generated: 0.39064986
Generated: 0.93411225
Sum: 2.3414445, Average: 0.4682889

```

图三十七 3.1 程序输出

(3.2). 编写 Java 应用程序实现如下功能：创建工作线程，模拟银行现金账户取款和存款操作。多个线程同时执行取款和存款操作时，如果不使用同步处理，会造成账户余额混乱，要求使用 `synchronized` 关键字同步代码块，以保证多个线程同时执行取款和存款操作时，银行现金账户取款和存款的有效和一致。要求采用实现 `Runnable` 接口和 `Thread` 类的构造方法的方式创建线程，而不是通过 `Thread` 类的子类的方式。在报告中附上程序截图（假设银行存款有 100 元，有 3 个取款线程和 2 个存款线程，每次取款和存款均为 10 元）、运行结果截图（显示每次存取款操作后的余额等信息，以说明线程间同步正确）和详细的文字说明。（10 分）

程序内容：编写 `BankAccount` 类，它表示一个银行账户，包含一个私有的 `balance` 变量，由题目设置初始值为 100。`BankAccount` 类有两个主要方法：`deposit` 和 `withdraw`。`deposit` 方法用于存款，使用同步块 `synchronized` 确保线程安全。每次存款后，更新余额并输出存款信息。`withdraw` 方法用于取款，同样使用同步块 `synchronized` 确保线程安全，在方法当中检查余额是否足够，若足够则更新余额并输出取款信息，否则输出余额不足的信息。

然后编写 `DepositTask` 类，实现 `Runnable` 接口，用于执行存款操作。编写构造方法接受一个 `BankAccount` 对象。编写 `run` 方法中循环 5 次，基于题目中的数据，设置每次存款 10 元，并暂停 100 毫秒以方便查看。`WithdrawTask` 类用于执行取款操作。构造方法接受一个 `BankAccount` 对象。编写 `run` 方法中循环 5 次，同理根据题目，设置每次取款 10 元，并暂停 100 毫秒以方便查看。

最后是主类，主方法当中创建一个 `BankAccount` 对象，并创建多个存款和取款线程，然后启动这些线程以模拟并发操作并输出结果。

程序：

```
1 package com.chino; // 定义包名
2
3 class BankAccount {
4     private int balance = 100; // 初始余额为 100
5
6     public void deposit(int amount) {
7         synchronized (this) { // 同步块，确保线程安全
8             balance += amount; // 增加余额
9             System.out.println(Thread.currentThread().getName() + " 存款 " + amount + " 元，当前余额: " + balance + " 元"); // 输出存款信息
10        }
11    }
12
13    public void withdraw(int amount) {
14        synchronized (this) { // 同步块，确保线程安全
15            if (balance >= amount) { // 检查余额是否足够
16                balance -= amount; // 减少余额
17                System.out.println(Thread.currentThread().getName() + " 取款 " + amount + " 元，当前余额: " + balance + " 元"); // 输出取款信息
18            } else {
19                System.out.println(Thread.currentThread().getName() + " 取款失败，余额不足!"); // 输出余额不足信息
20            }
21        }
22    }
23 }
24
25 class DepositTask implements Runnable {
26     private final BankAccount account; // 银行账户对象
27
28     public DepositTask(BankAccount account) {
29         this.account = account; // 初始化银行账户对象
30     }
31
32     @Override
33     public void run() {
34         for (int i = 0; i < 5; i++) { // 循环 5 次
35             account.deposit(amount:10); // 每次存款 10 元
36             try {
37                 Thread.sleep(millis:100); // 模拟操作时间，暂停 100 毫秒
38             } catch (InterruptedException e) {
39                 e.printStackTrace(); // 捕获并打印中断异常
40             }
41         }
42     }
43 }
44
45 }
```

图三十八 3.2 程序①


```

65 public class BankSimulation {
    Run | Debug | 运行 main | 调试 main.
66     public static void main(String[] args) {
67         BankAccount account = new BankAccount(); // 创建银行账户对象
68
69         Thread withdrawThread1 = new Thread(new WithdrawTask(account), name:"取款线程1"); // 创建取款线程1
70         Thread withdrawThread2 = new Thread(new WithdrawTask(account), name:"取款线程2"); // 创建取款线程2
71         Thread withdrawThread3 = new Thread(new WithdrawTask(account), name:"取款线程3"); // 创建取款线程3
72
73         Thread depositThread1 = new Thread(new DepositTask(account), name:"存款线程1"); // 创建存款线程1
74         Thread depositThread2 = new Thread(new DepositTask(account), name:"存款线程2"); // 创建存款线程2
75
76         withdrawThread1.start(); // 启动取款线程1
77         withdrawThread2.start(); // 启动取款线程2
78         withdrawThread3.start(); // 启动取款线程3
79         depositThread1.start(); // 启动存款线程1
80         depositThread2.start(); // 启动存款线程2
81     }
82 }

```

图三十九 3.2 程序②

输出结果:

```

取款线程1 取款 10 元, 当前余额: 90 元
取款线程3 取款 10 元, 当前余额: 80 元
存款线程2 存款 10 元, 当前余额: 90 元
存款线程1 存款 10 元, 当前余额: 100 元
取款线程2 取款 10 元, 当前余额: 90 元
存款线程1 存款 10 元, 当前余额: 100 元
存款线程2 存款 10 元, 当前余额: 110 元
取款线程1 取款 10 元, 当前余额: 100 元
取款线程3 取款 10 元, 当前余额: 90 元
取款线程2 取款 10 元, 当前余额: 80 元
存款线程1 存款 10 元, 当前余额: 90 元
存款线程2 存款 10 元, 当前余额: 100 元
取款线程3 取款 10 元, 当前余额: 90 元
取款线程1 取款 10 元, 当前余额: 80 元
取款线程2 取款 10 元, 当前余额: 70 元
存款线程1 存款 10 元, 当前余额: 80 元
存款线程2 存款 10 元, 当前余额: 90 元
取款线程3 取款 10 元, 当前余额: 80 元
取款线程1 取款 10 元, 当前余额: 70 元
取款线程2 取款 10 元, 当前余额: 60 元
存款线程1 存款 10 元, 当前余额: 70 元
存款线程2 存款 10 元, 当前余额: 80 元
取款线程3 取款 10 元, 当前余额: 70 元
取款线程1 取款 10 元, 当前余额: 60 元
取款线程2 取款 10 元, 当前余额: 50 元

```

图四十 3.2 程序输出

(3.3). 有一座南北向的桥，只能容纳一个人，桥的南边有 1000 个人（记为 S1,S2,...,S1000）和桥的北边有 1000 个人（记为 N1,N2,...,N1000），编写 Java 应用程序让这些人到达对岸，每个人用一个线程表示，桥为共享资源，在过桥的过程中输出谁正在过桥（不同人之间用逗号隔开）。运行 10 次，分别统计南边的 1000 人和北边的 1000 人先全部到达对岸的次数（第 i 行输出格式为：第 i 次运行，南边/北边先完成过桥）。要求采用实现 Runnable 接口和 Thread 类的构造方法的方式创建线程，而不是通过 Thread 类的子类的方式。在报告中附上程序截图、运行结果截图和详细的文字说明（包括对结果的分析）。（10 分）

程序内容：首先编写 BridgeCrossing 类，根据题目要求定义常量 NUM_PEOPLE 表示每边过桥的人数为 1000，并使用公平锁 ReentrantLock 确保线程按照请求锁的顺序依次获得锁。定义 southCount 和 northCount 分别记录南边和北边过桥的人数。在主方法 main 中，定义了 southWins 和 northWins 分别记录南边和北边先完成过桥的次数。程序进行 10 次模拟运行，每次运行前重置 southCount 和 northCount。接着，创建并启动南北两边各 1000 个线程，模拟过桥过程。使用 join 方法等待所有线程执行完毕后，根据 southCount 和 northCount 判断哪一边先完成过桥，并记录结果。最后输出 10 次运行中南北两边先完成过桥的次数。

然后编写内部类 Person 实现 Runnable 接口，定义每个人的编号和过桥方向。在 run 方法中，获取锁后模拟过桥过程（通过 Thread.sleep 进行模拟），并根据方向增加相应的计数器。使用 finally 块确保锁在过桥后被释放。由于使用了公平锁，线程获取锁的顺序与请求锁的顺序一致，这意味着南北两边的人过桥的机会是均等的。每次运行中，南北两边各 1000 人同时开始过桥，最终统计哪一边先完成过桥。运行 10 次后，最后输出南北两边先完成过桥的次数。

程序：

```
1 package com.chino;
2
3 import java.util.concurrent.locks.ReentrantLock;
4
5 public class BridgeCrossing {
6     private static final int NUM_PEOPLE = 1000; // 定义过桥人数
7     private static final ReentrantLock bridge = new ReentrantLock(fair:true); // 使用公平锁
8     private static int southCount = 0; // 记录南边过桥人数
9     private static int northCount = 0; // 记录北边过桥人数
10
11     Run | Debug | 运行 main | 调试 main.
12     public static void main(String[] args) {
13         int southWins = 0; // 记录南边过桥胜利次数
14         int northWins = 0; // 记录北边过桥胜利次数
15
16         for (int run = 1; run <= 10; run++) { // 进行10次运行
17             southCount = 0; // 每次运行前，将南边过桥人数和北边过桥人数重置为0
18             northCount = 0;
19
20             Thread[] southThreads = new Thread[NUM_PEOPLE]; // 创建南边过桥线程数组
21             Thread[] northThreads = new Thread[NUM_PEOPLE]; // 创建北边过桥线程数组
22
23             for (int i = 0; i < NUM_PEOPLE; i++) { // 创建南边和北边过桥线程
24                 southThreads[i] = new Thread(new Person("S" + (i + 1), direction:"South"));
25                 northThreads[i] = new Thread(new Person("N" + (i + 1), direction:"North"));
26             }
27
28             for (int i = 0; i < NUM_PEOPLE; i++) { // 启动南边和北边过桥线程
29                 southThreads[i].start();
30                 northThreads[i].start();
31             }
32         }
33     }
34 }
```

图四十一 3.3 程序①

```

32         try {
33             for (int i = 0; i < NUM_PEOPLE; i++) { // 等待南边和北边过桥线程执行完毕
34                 southThreads[i].join();
35                 northThreads[i].join();
36             }
37         } catch (InterruptedException e) {
38             e.printStackTrace();
39         }
40
41         String winner = southCount > northCount ? "南边" : "北边"; // 判断哪边先完成过桥
42         System.out.printf(format:"第%d次运行, %s先完成过桥%n", run, winner); // 输出哪边先完成过桥
43
44         if (southCount > northCount) { // 记录哪边先完成过桥的次数
45             southWins++;
46         } else {
47             northWins++;
48         }
49     }
50
51     System.out.printf(format:"南边先完成过桥次数: %d\n", southWins); // 输出南边先完成过桥的次数
52     System.out.printf(format:"北边先完成过桥次数: %d\n", northWins); // 输出北边先完成过桥的次数
53 }
54
55 static class Person implements Runnable {
56     private final String name; // 人的名字
57     private final String direction; // 过桥方向
58
59     public Person(String name, String direction) {
60         this.name = name;
61         this.direction = direction;
62     }
63
64     @Override
65     public void run() {
66         try {
67             bridge.lock(); // 获取锁
68             System.out.println(name + " 正在过桥 (" + direction + ")");
69             Thread.sleep(millis:0); // 模拟过桥时间
70             if (direction.equals(anObject:"South")) { // 如果是南边过桥
71                 southCount++;
72             } else { // 如果是北边过桥
73                 northCount++;
74             }
75         } catch (InterruptedException e) {
76             e.printStackTrace();
77         } finally {
78             bridge.unlock(); // 释放锁
79         }
80     }
81 }
82 }
83

```

图四十二 3.3 程序②

输出结果:

```

N885 正在过桥 (North)
N992 正在过桥 (North)
N976 正在过桥 (North)
N880 正在过桥 (North)
N856 正在过桥 (North)
S892 正在过桥 (South)
第10次运行, 北边先完成过桥
南边先完成过桥次数: 0
北边先完成过桥次数: 10

```

图四十三 3.3 程序输出

+++++

其他（例如感想、建议等等）。

通过这一次的实验更加熟悉 Java 相关的语法尤其是线程相关的，对于线程的性质有了更深刻的认知与熟练度。

其次是关于库类，本次实验当中使用了 HashMap 库类，确实对于稀疏矩阵的加乘运算的快捷有立竿见影的效果。

每一次运行线程结果的区别都很大，也深刻体会到了线程本身其实是较为底层的东西。

接着加油！

指导教师批阅意见：
成绩评定：
指导教师签字：
2024 年 月 日
备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。